PART 1: CORE COMMUNICATION PATTERNS

Sync vs Async Communication

Core Idea

In microservices, services communicate either:

- **Synchronous (Sync):** Immediate request-response
- **Asynchronous (Async):** Fire-and-forget via events/messages

📊 **Comparison Table**

Factor	Sync	Async
Latency	Low (immediate)	Higher (event delay)
Coupling	Tight	Loose
Scalability	Limited	High
Consistency	Strong	Eventual
Complexity	Simple	Complex
Use when	Immediate response needed	Processing can wait

📍 **Decision Tree**

text

Step 1: Immediate response required?

YES → Sync

NO → Step 2

Step 2: User-facing critical path?

YES → Sync

NO → Step 3

Step 3: Can processing be delayed?

YES → Async

NO → Sync

Step 4: High load/scalability needs?

YES → Async preferred

NO → Sync acceptable

Step 5: Strong consistency required?

YES → Sync

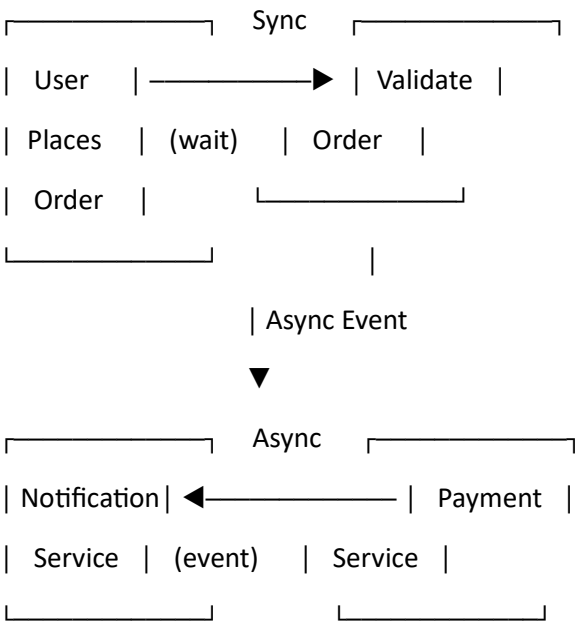
NO → Async

🧠 **Mental Model**

- **Sync** = Phone call (wait for reply)
- **Async** = Message/postbox (no waiting)

🎯 **Hybrid Example (Order Flow)**

text



PART 2: DATA MANAGEMENT

Data Decomposition

Core Principle

Database per service — each microservice owns its own database.

Why No Shared Database?

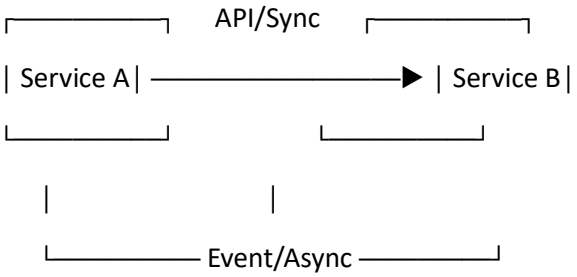
- Creates tight coupling
- Limits scalability
- Prevents independent deployment

Data Ownership Examples

Service	Data Owned
Order	order_id, items, order_status
Customer	customer_id, name, address
Payment	payment_id, order_id, payment_status
Shipping	shipment_id, tracking_status

🔄 **Communication Without Shared DB**

text



Key Challenge: Eventual Consistency

Solution Patterns:

- **Saga Pattern** for distributed transactions
- **CQRS** for separating read/write models
- **Event Sourcing** for maintaining consistency

🧠 **Mental Model**

- **Monolith** = one shared brain
- **Microservices** = multiple independent brains

PART 3: RESILIENCE PATTERNS

The Resilience Stack (Order of Protection)

text

Level 1: TIMEOUT ———▶ "Don't wait forever"

|

▼

Level 2: RETRY ————▶ "Try again if temporary"

|

▼

Level 3: CIRCUIT BREAKER —▶ "Stop calling failing service"

|

▼

Level 4: FALLBACK ———▶ "Provide alternative response"

1. Timeout

Aspect	Detail
What	Limits max wait time for a call
Rule	EVERY remote call needs timeout
Why	Prevents thread blocking, resource exhaustion

2. Retry Pattern

Aspect	Detail
When	Network glitches, temporary overload, 5xx errors
When NOT	4xx errors, non-idempotent operations
Best Practices	Exponential backoff + jitter + limited attempts

Retry Backoff Diagram

text

Attempt 1: Wait 100ms —▶ Fail

Attempt 2: Wait 200ms —▶ Fail

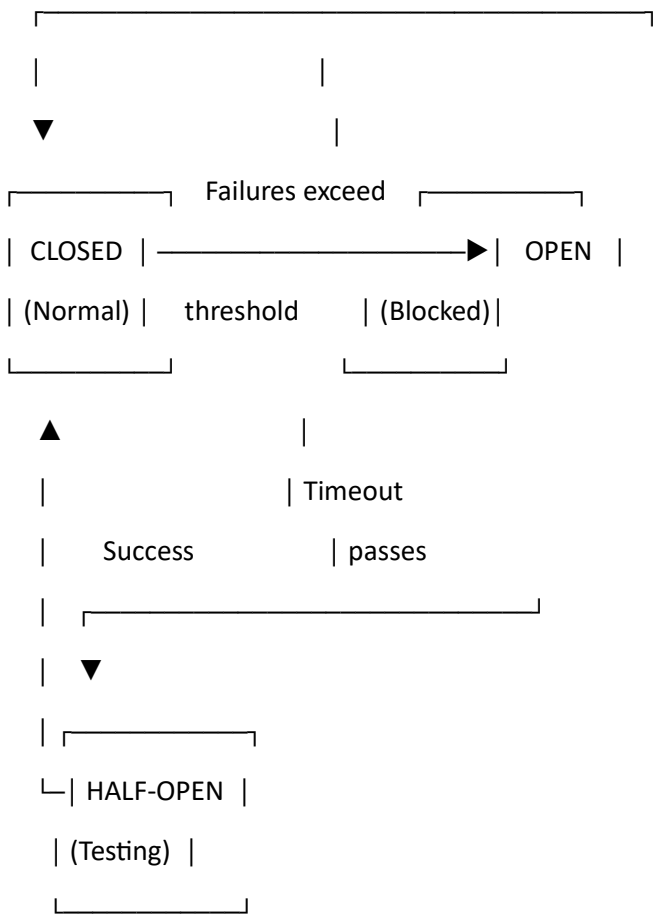
Attempt 3: Wait 400ms —▶ Success ✓

(Exponential backoff with jitter)

3. Circuit Breaker

States Flow

text



4. Fallback Pattern

Type	Example
Cached data	Serve stale cache when live API fails
Default response	"Service temporarily unavailable"
Secondary provider	Switch to backup service

🎯 **Real-World Example (Payment Flow)**

text

```
|                PAYMENT FLOW                    |
├────────────────────────────────────────────────┤
| 1. Timeout: 2 seconds max wait                 |
| 2. Retry: 2 attempts with exponential backoff  |
| 3. Circuit Breaker: Opens after 5 failures in 10 sec |
| 4. Fallback: Mark order as "payment pending"   |
└────────────────────────────────────────────────┘
```

PART 4: RATE LIMITING

What It Does

Controls number of requests a client/user/service can make within a time window.

Algorithms Comparison

Algorithm	Burst Allowed?	Complexity	Best For
Token Bucket	✓ Yes	Medium	Most production systems
Fixed Window	✗ No	Simple	Basic throttling
Sliding Window	Limited	High	Accurate rate limiting
Leaky Bucket	✗ No	Medium	Smooth output traffic

Token Bucket Explained

text

Tokens added at steady rate (10/sec)

```
      ↓
  ┌──────────────────┐
  | Token Bucket     |
  | Capacity: 50     |
  | ●●●●●●●●●●       |
  └──────────────────┘
      ↓
```

Each request consumes 1 token

↓

Burst: 50 requests can come instantly!

HTTP Status Code

429 Too Many Requests → Rate limit exceeded

🧠 **Mental Model**

Rate limiter = Traffic police (decides who goes, who waits, who stops)

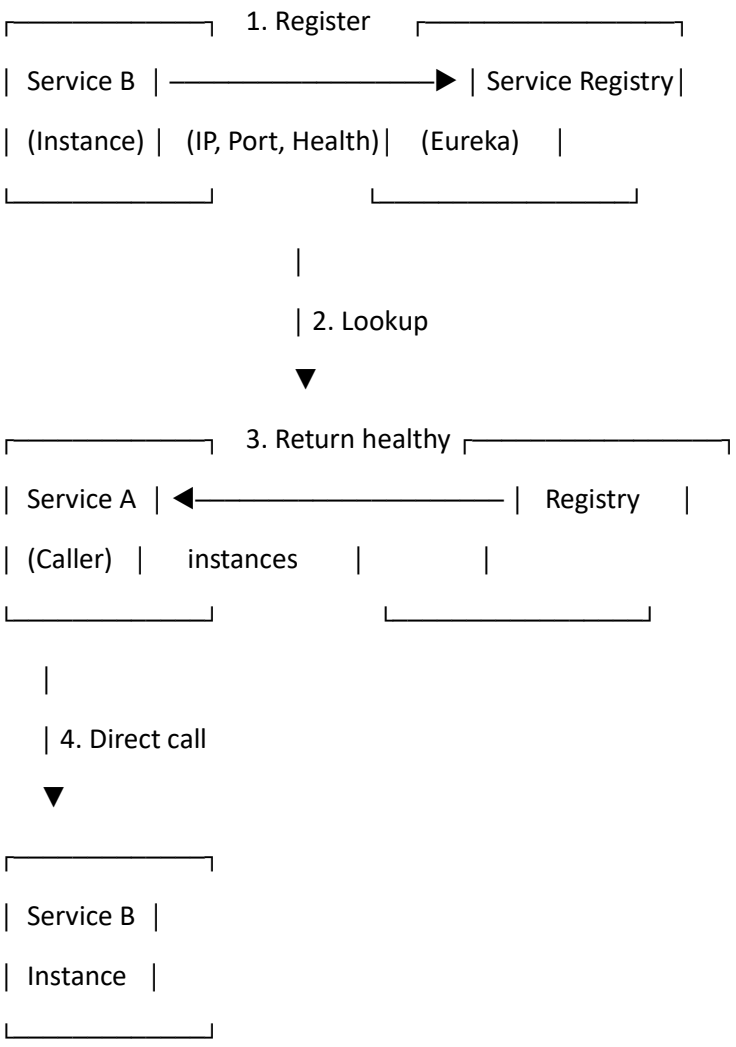
PART 5: SERVICE DISCOVERY

Definition

Mechanism that allows microservices to find and communicate with each other dynamically.

How It Works

text



Types

Type	Who Decides Routing	Example Tools
Client-Side	Client	Eureka + Ribbon
Server-Side	Load Balancer	Kubernetes, AWS ALB

🧠 **Mental Model**

- Service Discovery = "Google Maps for microservices"
- Registry = Directory of all services
- Load Balancer = Traffic controller

PART 6: SAGA PATTERN

Definition

Manages distributed transactions across multiple microservices without using a single ACID transaction.

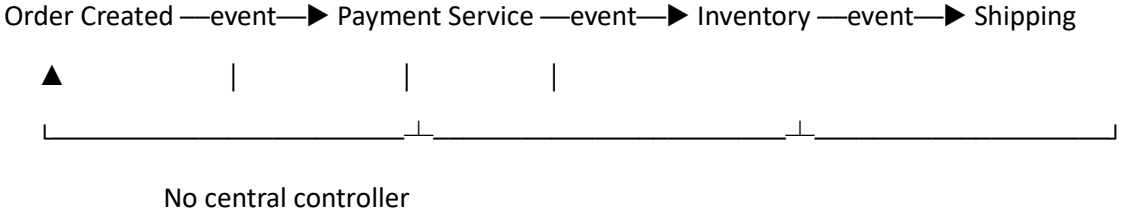
Why Needed

- No shared database
- No global ACID transaction support
- Failures are common

Types of Saga

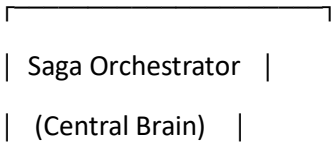
1. Choreography (Event-Driven)

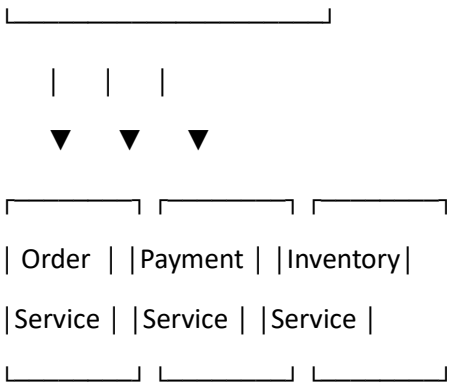
text



2. Orchestration (Central Controller)

text





Saga vs ACID

Feature	ACID Transaction	Saga Pattern
Scope	Single DB	Multiple services
Consistency	Strong	Eventual
Locking	Yes	No
Scalability	Limited	High
Failure handling	Rollback	Compensating actions

Compensating Transactions Example

text

Forward Actions: Compensating Actions:

1. Create Order → Cancel Order

2. Deduct Payment → Refund Payment

3. Reserve Stock → Release Stock

4. Create Shipment → Cancel Shipment

🧠 **Mental Model**

- **ACID** = Single chef in one kitchen
- **Saga** = Multiple chefs in different kitchens coordinating via messages

---

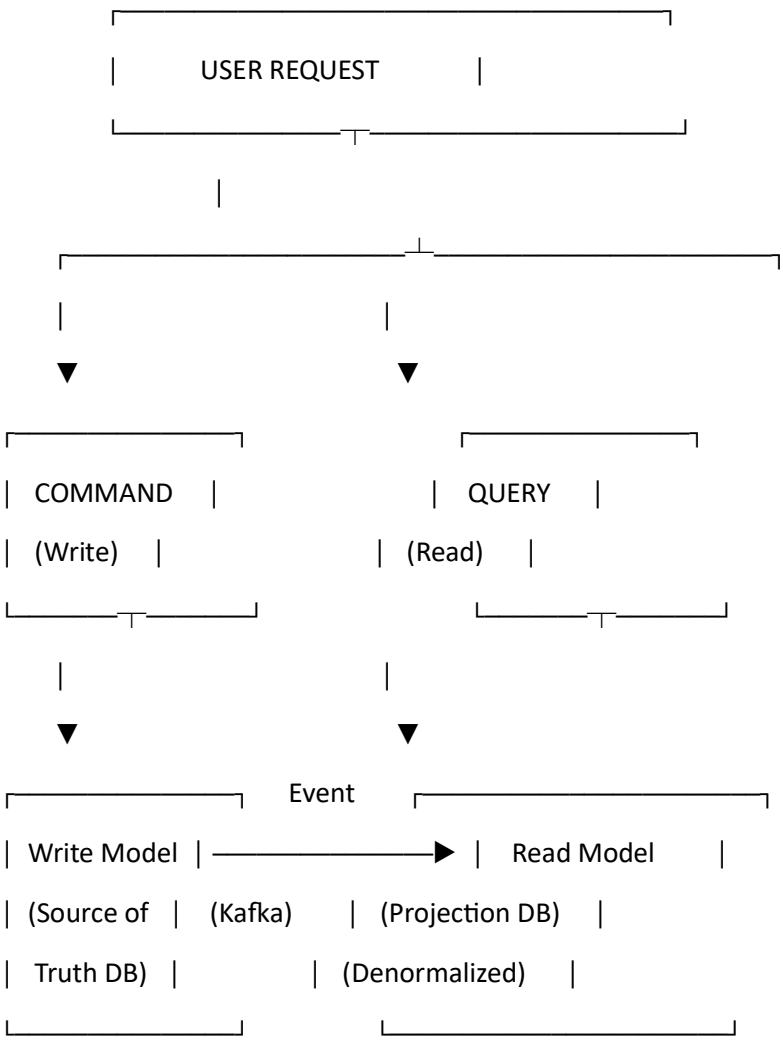
PART 7: CQRS & DATA DUPLICATION

Definition

CQRS = Command Query Responsibility Segregation

Core Architecture

text



Intentional Data Duplication

Write Model	Read Model
Normalized data	Denormalized data
Transactional focus	Query-optimized focus
Single source of truth	Multiple projections possible

Example: Order Data

Write Model (Normalized):

sql

orders: order_id, customer_id, status

order_items: order_id, product_id, quantity

Read Model (Denormalized):

sql

order_view: order_id, customer_name, product_names, order_status, total_amount

🧠 **Mental Model**

- Write Model = Official record ledger
- Read Model = Optimized display copy
- *Users never query the ledger directly*

PART 8: IDEMPOTENCY

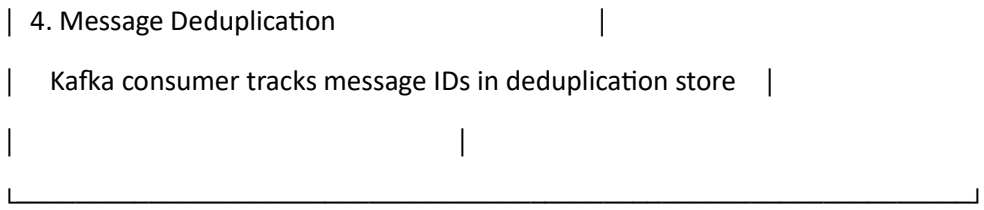
Definition

Executing the same operation multiple times produces the same result as executing it once.

Implementation Strategies

text

```
┌─────────────────────────────────────────────────────────────┐
│                IDEMPOTENCY IMPLEMENTATIONS                  │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│ 1. Idempotency Key                                          │
│    Client: POST /orders + Header: Idempotency-Key: abc123   │
│    Server: Store processed keys in Redis/DB                 │
│                                                             │
│ 2. Database Constraints                                     │
│    UNIQUE constraint on order_id, payment_id                │
│                                                             │
│ 3. State-based Updates                                      │
│    ✓ SET balance = 1000    (idempotent)                     │
│    ✗ ADD 100 to balance    (NOT idempotent)                 │
│                                                             │
```



Where Critical

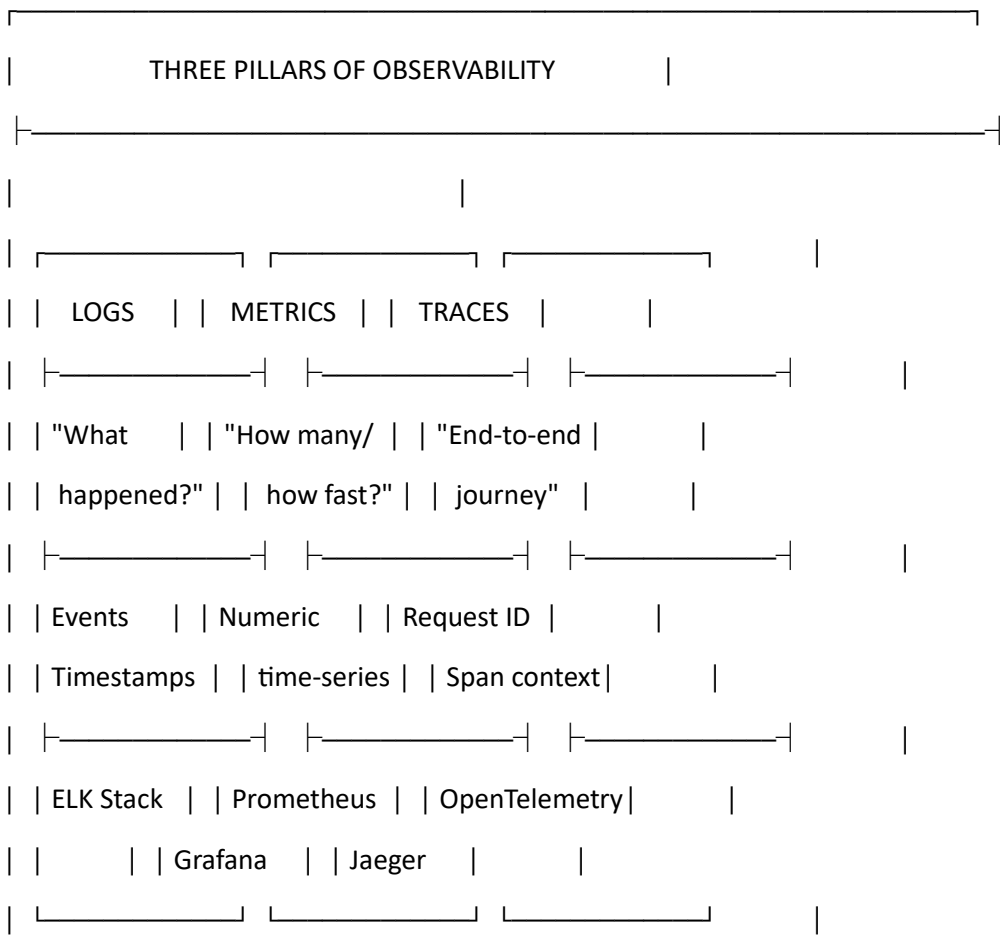
- Payment systems
- Order creation
- Inventory updates
- Event processing systems

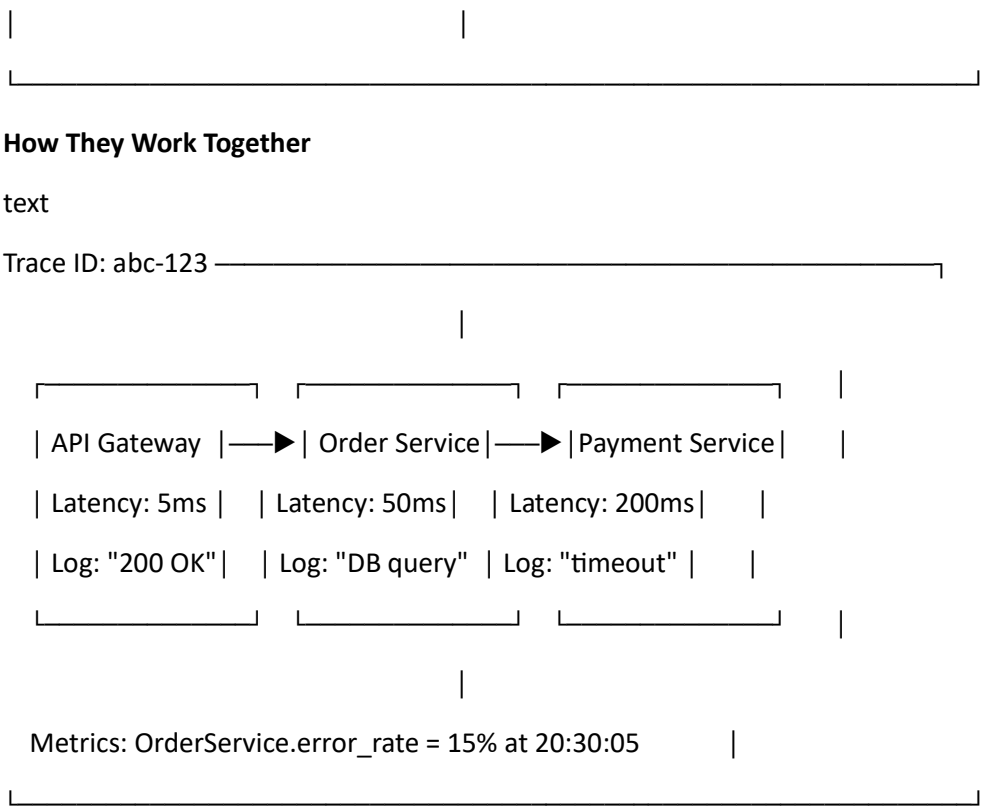
🧠 **Mental Model**

- **Idempotent** = Light switch (ON → ON → ON = same state)
- **Non-idempotent** = Adding coins (each action changes state)

PART 9: OBSERVABILITY (3 Pillars)

text





🧠 **Mental Model**

- **Logs** = Diary entries
- **Metrics** = Heartbeat monitor
- **Traces** = GPS route map

PART 10: CACHING STRATEGIES

Cache Aside (Lazy Loading) — MOST COMMON

text

Request —▶ Cache —Miss—▶ Database —▶ Update Cache —▶ Response

|

└—Hit—▶ Response (fast path)

Write-Through

text

Write —▶ Cache —Sync write—▶ Database —▶ Response

Write-Back (Write-Behind)

text

Write —▶ Cache —▶ Response (immediate)

|

└──Async──▶ Database (eventual)

Cache Invalidation Strategies

Strategy	How It Works	Best For
TTL	Data expires after time	Product catalogs
Event-based	Update cache on data change	User profiles
Manual	Explicit invalidation API	Admin-triggered updates

Cache Stampede Problem & Solutions

text

Problem: Cache expires → 1000 requests hit DB simultaneously

Solutions:

1. Locking (only 1 request fetches)
2. Request coalescing (deduplicate in-flight requests)
3. Random TTL jitter (stagger expiration times)

🧠 **Mental Model**

- **Cache** = Fast memory desk
- **Database** = Slow library
- *You keep notes on your desk instead of going to the library every time*

PART 11: DEPLOYMENT MODELS

Quick Comparison Matrix

Model	Downtime	Risk	Cost	Complexity	Use Case
Monolithic	High	High	Low	Low	Legacy systems
Rolling	None	Medium	Medium	Medium	Standard deployments

Model	Downtime	Risk	Cost	Complexity	Use Case
Blue-Green	None	Low	High	Medium	Critical apps
Canary	None	Very Low	Medium	High	Large-scale systems
Active-Passive	Minimal	Low	Medium	Low	DR systems
Active-Active	None	Low	High	Very High	Global high-scale apps

Visual Summary

text

Rolling: [▓▓▓▓▓▓▓] → [▓▓▓▓▓▒▒] → [▓▓▓▒▒▒▒] → [▓▒▒▒▒▒▒] → [▒▒▒▒▒▒▒]

(Gradual instance replacement)

Blue-Green: [▓▓▓▓▓▓] BLUE [▓▓▓▓▓▓] GREEN

\ /

└──Switch──┘

(Instant traffic cutover)

Canary: [▓▓▓▓▓▓▓▓▓▓▓▓▓] [▒▒] 5% traffic

Main version New version

(Gradual traffic increase)

Active-Active: 🌍 US ┐

├— All active, load balanced

🌍 EU ┘

📋 **ULTRA CHEAT SHEET (1-Page Revision)**

CORE PATTERNS MAP

text

QUICK REFERENCE CARD

Pattern	One-Line Summary
Sync	Immediate response, tight coupling, user-facing ops
Async	Event-driven, loose coupling, scalable processing
Saga	Distributed transactions via compensating actions
CQRS	Separate read/write models with intentional duplication
Circuit Breaker	Stop calling failing services, prevent cascading failures
Retry	Handle transient failures with exponential backoff
Timeout	Never wait indefinitely for any remote call
Idempotency	Same operation multiple times = same result
Rate Limiter	Control request flow, return 429 when exceeded

MENTAL MODEL CHEAT SHEET

text

API Gateway = Front door of the system

Rate Limiter = Security guard at entrance

Service Registry = Phone directory of services

Kafka = Postal system (async messages)

Circuit Breaker = Emergency electrical switch

Saga = Workflow conductor/orchestrator

Cache = Sticky notes on your desk

Trace = GPS tracker of a request

Idempotency Key = Unique receipt ID for each operation

INTERVIEW POWER PHRASES

"Microservices are independently deployable services, each owning its own data model aligned to business domains."

"We use synchronous communication for real-time user-driven flows, and asynchronous for scalability and decoupling."

"The resilience stack is timeout first, then retry, then circuit breaker, finally fallback."

"Saga replaces ACID with eventual consistency using compensating transactions."

"CQRS intentionally duplicates data — that's the trade-off for scalability and performance."

"Idempotency is essential for safe retries in distributed systems."

This document now serves as a complete interview preparation guide with:

- ✅ Clean formatting with emojis and visual hierarchy
- ✅ Text-based diagrams for easy recall
- ✅ Comparison tables for quick reference
- ✅ Mental models for intuitive understanding
- ✅ One-line summaries for interview delivery